

Universidad de los Andes

Ingeniería de Sistemas y Computación

ISIS-1611: Inteligencia Artificial

Semestre 2026-2

Taller 2: Búsqueda local y juegos adversarios

Isabela Archbold -202420166

Isabella Largacha - 2024020593

Samuel Zuluaga -202520012

Juan José Castro Maldonado -202520030

Juan David Forero - 202411707

18 de septiembre de 2026

Tabla de contenido

1. Ascenso de colina

- 1.1. Formulación del problema de optimización
- 1.2. Implementación de configuration_score
- 1.3. Implementación de hill_climbing
- 1.4. Experimentos
- 1.5. Interpretación

2. Recocido simulado

- 2.1 Implementación de cooling_schedule
- 2.2 Implementación de simulated_annealing
- 2.3 Comparación de parámetros (To, alpha)

3. Algoritmo genético

- 3.1 Implementación de one_point_crossover
- 3.2 Implementación de swap_mutation
- 3.3 Implementación de genetic_algorithm
- 3.4 Comparación de configuraciones
- 3.5 Interpretación

4. Minimax

- 4.1 Formulación formal del juego
- 4.2 Implementación de MinimaxAgent.get_action
- 4.3 Convención de profundidad y conteo de nodos
- 4.4 Experimentos de profundidad
- 4.5 Interpretación

5. Alfa-beta y función de evaluación

- 5.1 Implementación de Alfa Beta y función evaluación:
- 5.2 Diseño de evaluation_function
- 5.3 Equivalencia y comparación de nodos
- 5.4 Justificación de función de evaluación

6. Análisis

- 6.1 Evidencia comparativa de optimización
- 6.2 (a) Características de los layouts que explican las diferencias
- 6.3 (b) Efecto de inicialización, aleatoriedad y parámetros
- 6.4 (c) Compromiso calidad vs. costo computacional
- 6.5 (d) Caso donde la profundidad cambia la decisión
- 6.6 (e) Minimax = alfa-beta, profundidad y orden de acciones

7. Reflexión sobre el uso de IA

- 7.1. Isabela Archbold
- 7.2. Juan David Forero
- 7.3. Samuel Zuluaga:
- 7.4. Juan José Castro Maldonado
- 7.5. Isabella Largacha

1. Ascenso de colina

1.1. Formulación del problema de optimización

Elemento	Definición en el proyecto
Representación de una solución	Una solución es una tupla de 0 y 1. Cada posición representa uno de los sitios candidatos. 1 significa que se coloca un módulo en ese sitio y 0 significa que no.
Restricción de validez	La configuración debe tener una posición por cada sitio candidato, todos los valores deben ser 0 o 1, y debe tener exactamente el número de módulos indicado por <code>module_count</code> .
Función objetivo	Se busca maximizar cobertura - redundancia - exposición. La cobertura aumenta el puntaje, mientras que la redundancia y la exposición lo disminuyen.
Relación de vecindad	Un vecino se obtiene quitando un módulo de uno de los sitios ocupados y colocándolo en otro sitio que estaba vacío. Así se mantiene el mismo número de módulos.

1.2. Implementación de `configuration_score`

La función obtiene de `score_components` tres valores: cobertura, redundancia y exposición, en ese orden. La cobertura es positiva porque queremos aumentarla. La redundancia y la exposición son penalizaciones, por lo que se restan. La implementación usada fue:

```
coverage, redundancy, exposure = problem.score_components(configuration)

return coverage - redundancy - exposure
```

1.3. Implementación de `hill_climbing`

El algoritmo empieza con la configuración inicial y calcula su puntaje. Después revisa todos los vecinos posibles y escoge el que tenga el mayor puntaje. Solo acepta el vecino si tiene un puntaje estrictamente mayor que el actual. Si dos vecinos tienen el mismo puntaje, se mantiene el primero porque los vecinos son revisados en el orden en que los entrega `problem.neighbors(current)`. La búsqueda termina cuando no encuentra ningún vecino que mejore el puntaje o cuando llega al máximo de iteraciones.

La configuración inicial no se modifica directamente. Se usa como el estado actual y las nuevas configuraciones se guardan por separado en el historial. Al final se retorna un `OptimizationResult` con la configuración final, su puntaje, el número de evaluaciones, el número de iteraciones y los historiales de configuraciones y puntajes.

1.4. Experimentos

Las ejecuciones se realizaron desde la interfaz de terminal del proyecto, invocada con `python3 main.py`. En el menú se seleccionó visualización gráfica, velocidad de animación normal, perfil de búsqueda equilibrado, una repetición y la semilla correspondiente a cada configuración inicial, y finalmente la opción Ejecutar selección.

#	Layout	Config. inicial	Puntaje Inicial	Puntaje Final	Iteraciones	Evaluaciones	¿Quedó por debajo del mejor observado?
1	local_peak	seed 7	61.00	96.00	2	46	No
2	local_peak	seed 12	67.00	81.00	1	31	Sí
3	local_peak	seed 33	81.00	81.00	0	16	Sí
4	plateau_grid	seed 7	50.00	92.00	1	31	Sí
5	plateau_grid	seed 12	56.00	92.00	2	46	Sí
6	plateau_grid	seed 33	71.00	92.00	1	31	Sí
7	plateau_grid	seed 50	50.00	92.00	1	31	Sí
8	plateau_grid	seed 77	70.00	92.00	1	31	Sí
9	plateau_grid	seed 100	83.00	94.00	1	31	No

Evidencia:

Figura 1. local_peak, semilla 7

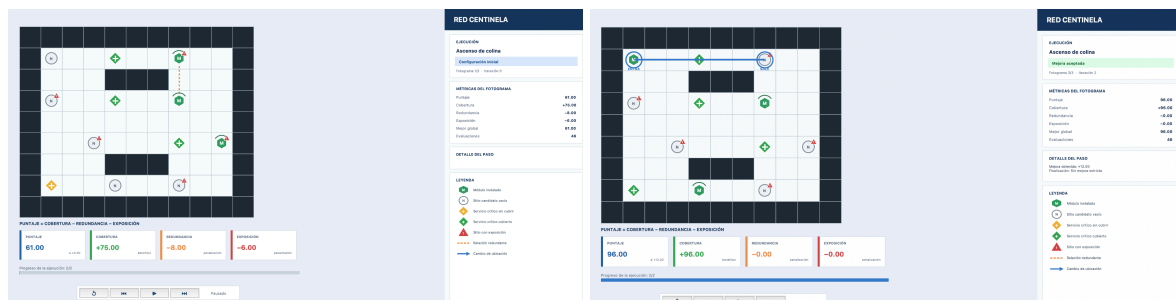


Figura 2. local_peak, semilla 12

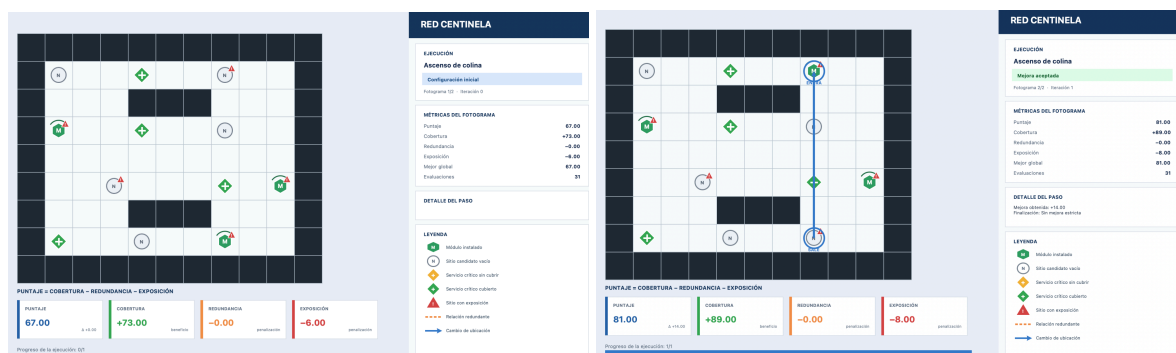


Figura 3. local_peak, semilla 33, 0 iteraciones

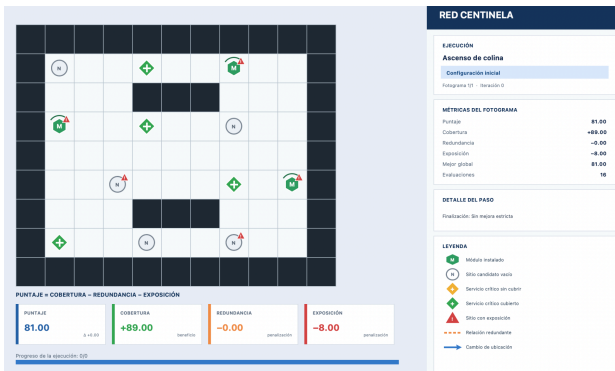


Figura 4. plateau_agrid, semilla 7

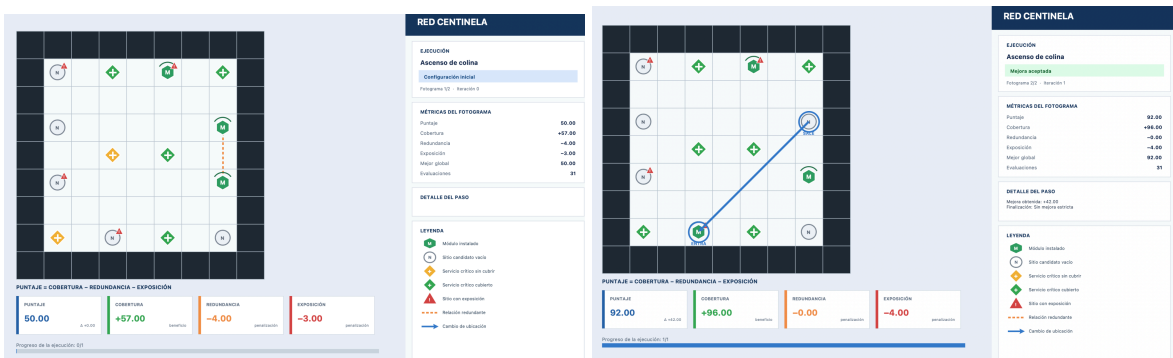


Figura 5. plateau_agrid, semilla 12

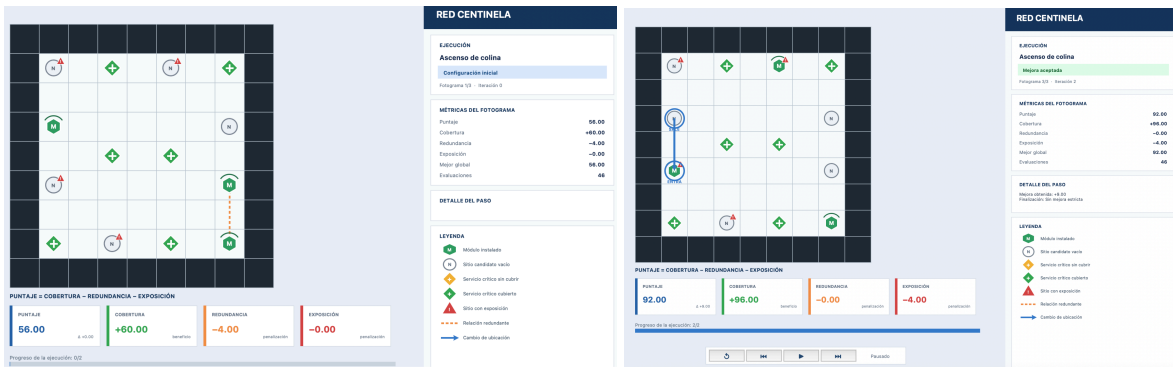


Figura 6. plateau_grid, semilla 33

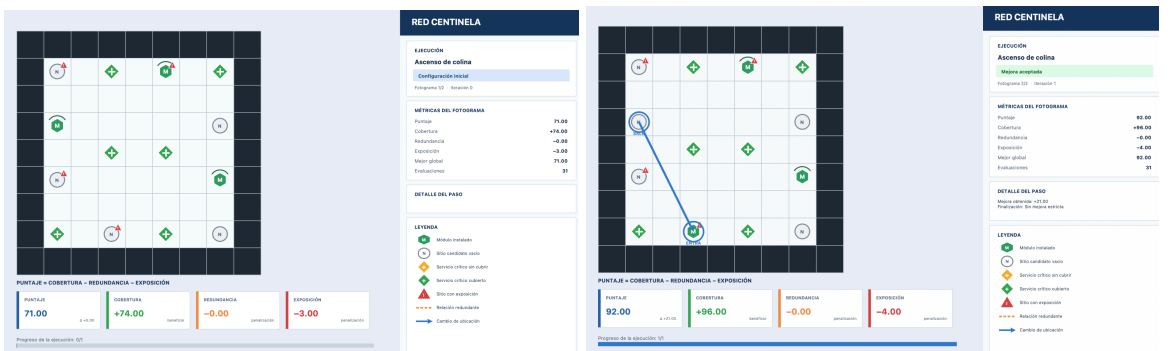


Figura 7. plateau_grid, semilla 50



Figura 8. plateau_grid, semilla 77

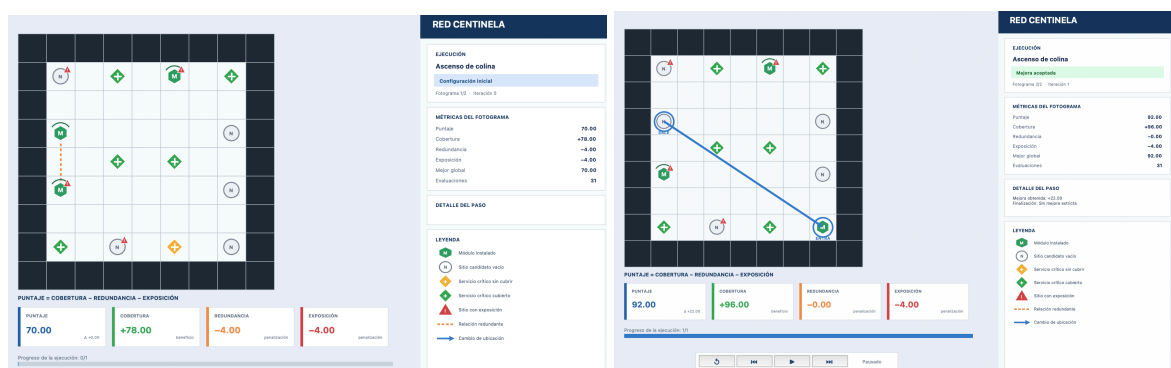
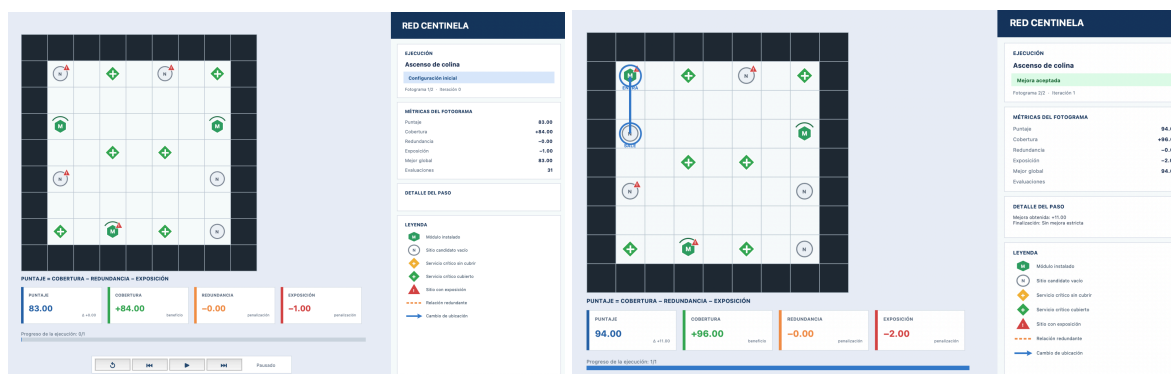


Figura 9. plateau_grid, semilla 100



1.5. Interpretación

Los experimentos muestran que el resultado de hill climbing depende de la configuración inicial. Esto se ve especialmente en local_peak, puesto que con la seed 7 el algoritmo pasó de 61.00 a 96.00, mientras que con la seed 12 pasó de 67.00 a 81.00. Con la seed 33 empezó directamente en 81.00 y no hizo ninguna mejora. Por lo tanto, diferentes puntos de inicio pueden llevar al algoritmo a diferentes resultados. El caso de la seed 33 es un ejemplo claro de un óptimo local. El algoritmo hizo 16 evaluaciones y tuvo 0 iteraciones, porque revisó todos sus vecinos y ninguno tenía un puntaje mayor que 81.00. Sin embargo, en otro experimento se encontró un resultado de 96.00. Esto muestra que detenerse en un óptimo local no significa haber encontrado el mejor resultado posible.

En `plateau_grid` se observa otro comportamiento. Varias semillas llegaron a 92.00, aunque empezaron con puntajes diferentes y necesitaron diferentes cantidades de evaluaciones. Sin embargo, la seed 100 llegó a 94.00. Esto también ayuda a entender el comportamiento de una meseta. Una meseta es una zona donde diferentes configuraciones pueden tener el mismo puntaje. Como el algoritmo solo acepta mejoras estrictas usando `>`, no acepta moverse a un vecino que tenga exactamente el mismo puntaje. Por eso puede quedarse en una zona de 92.00 aunque existan otras configuraciones con ese mismo valor o incluso caminos que lleven a valores mayores.

2. Recocido simulado

2.1 Implementación de `cooling_schedule`

```
def cooling_schedule(initial_temperature: float, cooling_rate: float, iteration: int) -> float:
    """
    Retorna el programa geométrico  $T(t) = T_0 * \alpha^{**t}$ .
    """
    return initial_temperature * (cooling_rate ** iteration)
```

La función de `cooling_schedule` calcula la temperatura del algoritmo en una iteración específica. El programa de enfriamiento retorna la temperatura inicial de esa iteración y la multiplica por la tasa de enfriamiento elevada al número de la iteración que vaya el algoritmo. Ya que la tasa de enfriamiento es menor a uno, por cada iteración que pasa, el retorno se vuelve más pequeño, siendo cada vez más menos permisivo con el fitness.

Esta función se invoca en cada iteración del loop principal para obtener la temperatura actual de ese momento. Posteriormente se usa la regla de aceptación. Una temperatura alta hará que el algoritmo sea mucho más permisivo para priorizar la exploración. En cambio cuando la temperatura sea menor, hará que la permisividad baje considerablemente para favorecer la explotación.

2.2 Implementación de `simulated_annealing`

En cada iteración del `while`, el cual se repite en el número de iteraciones máximas, se genera la lista completa de vecinos con la misma operación que se usa en ascenso de colina, solo que en este algoritmo solo se elige un vecino al azar. Al pasar por la instancia `rgn.choice`, se garantiza reproducibilidad, de que si se llama a la función dos veces con la misma semilla en `rng`, el resultado sea el mismo. Se calcula el delta de energía con respecto a la iteración anterior, si el delta es mayor a cero, se acepta sin necesidad de evaluar probabilidad. En el caso contrario, donde el delta es menor a cero, se acepta con probabilidad comparando este valor con un número aleatorio uniforme. Esta regla le da al recocido simulado la capacidad de escapar de óptimos locales con el fin de buscar

óptimos mejores. El algoritmo acaba bajo dos condiciones: que el número de iteraciones se agote, o que se llegue a la temperatura mínima.

2.3 Comparación de parámetros (To, alpha)

- **Config 1:** tiene una temperatura inicial alta y un alfa cercano a 1. Esto resultó en un puntaje medio más alto que el de la configuración siguiente. Esta configuración casi siempre encuentra el óptimo o algo muy cercano. Sin embargo, tarda en hacerlo porque el enfriamiento es tan lento que nunca llega a apagarse en el límite.

Config	Corrida	Semilla	Puntaje final	Mejor puntaje visitado
Config 1	1	1	115	115
Config 1	2	2	116	116
Config 1	3	3	116	116
Config 1	4	4	116	116
Config 1	5	5	116	116

- **Config 2:** tiene una temperatura inicial baja y un alfa más agresivo. Llegó a un resultado de manera más rápida pero este es mucho menos confiable, ya que tiene una desviación estándar mayor. En ocasiones, la configuración causa que el algoritmo pare o se quede atrapado sin poder llegar a óptimos locales.

Config	Corrida	Semilla	Puntaje final	Mejor puntaje visitado

config 2	1	1	116	116
config 2	2	2	113	113
config 2	3	3	108	108
config 2	4	4	116	116
config 2	5	5	113	113

Config	To	alpha	Layout	n corridas	Mejor puntaje	Puntaje medio	Desv . est.	Iteraciones medias	Evaluaciones medias
Config 1	33	0.99	risk_corridor	5	116	115,8	0.45	750	751
Config 2	5	0.90	risk_corridor	5	116	113.20	3.27	212	213

2.4. Interpretación

Los resultados finales de las dos configuraciones muestran dos escenarios con prioridades distintas, lo cual lleva a una calidad, consistencia y costo computacional variados. Una temperatura inicial alta con enfriamiento lento favorece casos donde es necesario explorar mucho o donde existe memoria computacional suficiente para satisfacer la capacidad requerida. utilizando esta configuración se llegará a soluciones más consistentes y cercanas a un óptimo. Por otro lado, una temperatura inicial baja con enfriamiento más agresivo favorece la explotación temprana de soluciones, es mucho más barata computacionalmente pero sacrifica la consistencia de las soluciones dadas. En un escenario con suficiente memoria computacional, es ideal realizar la configuración 1.

3. Algoritmo genético

3.1 Implementación de one_point_crossover

One point crossover usa dos padres de la misma longitud. Primero valida que sus longitudes coincidan, si no lanza un error, y si el cromosoma tiene menos de dos posiciones no es posible hacer la separación, por lo cual se retornan nuevamente los mismos padres.

Como solución general, se escogió usar `rng.randint(1, n-1)` para garantizar un punto de corte interior (nunca 0 o la longitud total). Adicionalmente, cada descendiente se armo combinando el prefijo del padre hasta el punto de corte definido y como sufijo combinando el otro padre desde ese mismo punto.

En python:

```
n = len(parent1)
```

```
c = rng.randint(1, n - 1)
```

```
child1 = parent1[:c] + parent2[c:]
```

```
child2 = parent2[:c] + parent1[c:]
```

```
return child1, child2
```

Esta solución no repara los descendientes ni valida los bits activos, pero esto se puede solucionar en `genetic_algorithm` (explicado luego).

3.2 Implementación de `swap_mutation`

Para `swap mutation`, la mutación se decide una sola vez después de leer el cromosoma completo, apoyándonos de la función `rng.random() < mutation_probability`. Así se determina si ese individuo va a mutar. Si no tiene que mutar, pues se retorna el mismo individuo.

Por otro lado, cuando sí tiene que mutar, primero separamos los índices activos (osea los que sean 1) de los inactivos, con un simple recorrido por el cromosoma. Si el cromosoma es de todo ceros o todo de unos, no hay ningún posible intercambio y se retorna el individuo original (es un caso poco probable pero puede suceder).

Si ambos grupos tienen al menos un elemento (0 o 1), se elige aleatoriamente un índice activo y uno inactivo con la función `rng.choice()`, y se intercambian sus valores pero en una lista copia del cromosoma (el original nunca lo tocamos).

En python:

```
if rng.random() < mutation_probability:
```

```
    indices_unos = [i for i, bit in enumerate(individual) if bit == 1]
```

```
    indices_ceros = [i for i, bit in enumerate(individual) if bit == 0]
```

```
if not indices_unos or not indices_ceros:
```

```
    return individual
```

```
cromosoma = list(individual)
```

```
i = rng.choice(indices_unos)
```

```
j = rng.choice(indices_ceros)
```

```
cromosoma[i] = 0
```

```
cromosoma[j] = 1
```

```
return tuple(cromosoma)
```

```
else:
```

```
    return individual
```

Al intercambiar un módulo de sitio (cada configuración tiene siempre exactamente k módulos instalados), la cantidad total de módulos nunca cambia, así que no toca reparar el cromosoma después de mutar. Distinto a lo que hace el cruce, donde combinar partes de dos padres si puede afectar cuantos módulos tiene el hijo.

3.3 Implementación de genetic_algorithm

El algoritmo empieza con una población inicial usando la función `problem.initial_population()` y calculando el puntaje de cada individuo con `configuration_score`. Usando esos puntajes guarda el mejor individuo hasta ese momento con `mejor_global`, para así poder retornar el mejor de toda la ejecución y no solo de la última generación.

Para cada generación se sigue el mismo ciclo.

1. Elitismo: ordena la población actual por puntaje (descendente) y se copian a la nueva población los `elite_size` mejores individuos, sin ningún cambio.
2. Reproducción: si la nueva población no llega al tamaño requerido, se seleccionan dos padres por torneo con `problem.tournament_select` y se cruzan con `one_point_crossover`, reparando cada hijo con `problem.repair_configuration`, para obligarlos a que vuelvan a tener k bits activos, y luego aplicamos `swap_mutation`.
3. Al terminar la nueva población, se reemplaza la anterior, se recalculan los puntajes y se revisa si el mejor individuo de esta generación es mejor al global registrado, si lo es se actualiza.

Todos estos pasos probabilísticos o aleatorios (estocásticos) reciben la misma fuente `rng` que entra como parámetro a la función (no usamos `random`). Al final se retorna un `OptimizationResult` con el

mejor individuo global, su puntaje, el total de evaluaciones acumuladas, el número de generaciones ejecutadas y los historiales de configuraciones y puntajes del mejor global por generación.

3.4 Comparación de configuraciones

Configuración	Población	Generaciones	Mutación	Elite	Layout	n corridas	Mejor	Medio	Desv. est	Evaluaciones	Generación del mejor
A	60	1	0.05	2	balanced_city	8	283.00	276.50	4.56	120.0	1
B	60	8	0.05	2	balanced_city	8	283.00	281.88	1.17	540.0	2
C	60	150	0.05	2	balanced_city	8	283.00	281.88	1.17	9060.0	2

Para aislar el efecto de un solo parámetro, se mantuvieron fijos la población (60), la probabilidad de mutación (0.05), el tamaño de élite (2) y el layout (balanced_city) en las tres configuraciones, variando únicamente el número de generaciones (1, 8 y 150).

Comandos:

- `python main.py -m optimization -a geneticAlgorithm -l balanced_city --population 60 --generations 1 --mutation 0.05 --elite 2 -n 8 --seed 7 -q`
- `python main.py -m optimization -a geneticAlgorithm -l balanced_city --population 60 --generations 8 --mutation 0.05 --elite 2 -n 8 --seed 7 -q`
- `python main.py -m optimization -a geneticAlgorithm -l balanced_city --population 60 --generations 150 --mutation 0.05 --elite 2 -n 8 --seed 7 -q`

3.5 Interpretación

El salto grande está entre A y B, donde pasamos de 1 a 8 generaciones. Ya que el promedio sube de 276.50 a 281.88 y la desviación cae de 4.56 a 1.17. Esto muestra que con mínimas generaciones, el resultado depende bastante de que tan buena haya sido la población inicial, pero con unas pocas generaciones más ya se puede solucionar esto.

Por otro lado, entre B y C donde pasamos de 8 a 150 generaciones, no hay ninguna mejora, solamente el gasto computacional incrementa bastante, de 540 a 9060 evaluaciones. Lo cual muestra que puede haber rendimientos decrecientes, donde después de un número de generaciones, tanta iteración no cambia nada. Esto se puede ver en la columna de generación del mejor, donde en ambas el mejor individuo aparece solo en la generación 2, el resto de generaciones no encontraron uno mejor.

Esto ocurre por la parte del elitismo en el ciclo, donde `elite_size=2` conserva siempre a los mejores, así que una vez la población pasa a una región buena del espacio de búsqueda, sin nueva diversidad la población deja de explorar y las generaciones adicionales solo repiten y llegan al mismo óptimo ya encontrado. Este mismo hallazgo, el de que la calidad permanece estable después de un número de

evaluaciones, lo vamos a retomar en el punto 6.4 para analizar el compromiso entre calidad de la solución y costo computacional.

4. Minimax

4.1 Formulación formal del juego

Componente	Definición en el proyecto
Estado inicial	El estado inicial está compuesto por el mapa la posición inicial del defensor, la posición inicial del intruso, el conjunto de terminales críticas pendientes, un puntaje inicial de 0 y 0 turnos transcurridos
Jugador en turno	En la raíz juega el defensor, correspondiente al agente 0 y al jugador MAX. Después juega el intruso, agente 1 y jugador MIN. Los dos agentes alternan durante todo el árbol
Acciones	Las acciones posibles son North, South, East, West y Stop, siempre que la posición resultante sea válida y no corresponda a un muro. Las acciones se exploran en el orden retornado por <code>state.get_legal_actions</code>
Función de resultado / sucesor	<code>state.generate_successor(agent_index, action)</code> crea un nuevo estado después de ejecutar una acción. Cuando se mueve el defensor se descuenta un punto y se suman 100 puntos si activa una terminal pendiente y cuando se mueve el intruso se incrementa el número de turnos
Prueba de estado terminal	El estado terminal es victoria cuando no quedan terminales pendientes y el defensor no fue interceptado y es derrota cuando el defensor y el intruso ocupan la misma posición.
Utilidad	Las victorias tienen utilidad +1000 y las derrotas -1000. En los estados no terminales alcanzados por el corte de profundidad se utiliza <code>evaluation_function</code> , que valora el puntaje, las terminales pendientes, la distancia del defensor al objetivo y el riesgo asociado a la cercanía del intruso.

4.2 Implementación de `MinimaxAgent.get_action`

La implementación considera al defensor como el agente 0 y jugador MAX, mientras que el intruso corresponde al agente 1 y jugador MIN. La función auxiliar recursiva alterna entre los agentes durante la exploración del árbol. Antes de expandir un estado se verifica si es victoria, derrota o si se alcanzó el límite de profundidad en cualquiera de estos casos se retorna el resultado de `evaluation_function`. Para cada acción legal se genera un sucesor mediante

state.generate_successor, respetando el orden entregado por state.get_legal_actions. MAX conserva el mayor valor encontrado y MIN conserva el menor. En caso de empate se mantiene la primera acción porque la acción seleccionada solamente se reemplaza cuando aparece un valor mejor

4.3 Convención de profundidad y conteo de nodos

La profundidad se mide en plies donde cada acción de cualquiera de los agentes consume un nivel. La raíz se encuentra en profundidad cero y no consume un ply. Por lo tanto, una profundidad de 1 incluye únicamente una acción del defensor, mientras que una profundidad de 2 incluye la acción del defensor y la respuesta del intruso. nodes_evaluated se reinicia al comenzar cada llamada a get_action y se incrementa una vez por cada estado procesado. El conteo incluye la raíz, los estados internos, los estados terminales y los estados donde se aplica el corte por profundidad.

4.4 Experimentos de profundidad

Layout	Profundidad	Acción inicial	Resultado de la partida	Rondas	Nodos (primera decisión)	Nodos (acumulado partida)
single_terminal	2	East	Victoria	7	13	112
bottleneck	2	Stop	Derrota	2	9	27
bottleneck	4	West	Victoria	21	89	2192

```
(.venv) isabellalargacha@MacBook-Air-de-Isabella-35 RedCentinela % python main.py -m adversarial -a MinimaxAgent -l single_terminal -d 2 -q
MinimaxAgent | profundidad=2 plies | acción_inicial=East | resultado=victoria | finalización=activación completa | nodos=112 | nodos_intruso=73 |
tiempo=0.001362s
(.venv) isabellalargacha@MacBook-Air-de-Isabella-35 RedCentinela % python main.py -m adversarial -a MinimaxAgent -l bottleneck -d 2 -q
MinimaxAgent | profundidad=2 plies | acción_inicial=Stop | resultado=derrota | finalización=intercepción | nodos=27 | nodos_intruso=19 | tiempo=0.
000253s
(.venv) isabellalargacha@MacBook-Air-de-Isabella-35 RedCentinela % python main.py -m adversarial -a MinimaxAgent -l bottleneck -d 4 -q
MinimaxAgent | profundidad=4 plies | acción_inicial=West | resultado=victoria | finalización=activación completa | nodos=2192 | nodos_intruso=1395
| tiempo=0.012659s
(.venv) isabellalargacha@MacBook-Air-de-Isabella-35 RedCentinela %
```

Ejecución de Minimax en single_terminal y comparación de profundidades 2 y 4 en bottleneck. Con profundidad 2 el agente seleccionó Stop y perdió por intercepción con profundidad 4 seleccionó West y obtuvo una victoria por activación completa.

4.5 Interpretación

En single_terminal Minimax seleccionó east y tuvo la victoria lo cual permitió comprobar el funcionamiento del algoritmo. En bottleneck al aumentar la profundidad de 2 a 4 piles se cambió la decisión final de stop a West. Con profundidad 2 la búsqueda termina en derrota por intercepción pero con la profundidad 4 se anticipó una secuencia más larga y se dio la victoria. Esta mejora tuvo un costo computacional donde: los nodos evaluados por el defensor aumentaron de 27 a 2.192. El experimento muestra que una búsqueda más profunda puede tomar una decisión de mejor calidad, pero necesita explorar un árbol mucho mayor.

5. Alfa-beta y función de evaluación

5.1 Implementación de Alfa Beta y función evaluación:

Implementación de AlphaBetaAgent:

Para implementar AlfaBeta, se definió una función llamada buscar dentro de get_action. Esta recibe el estado, el agente que tiene el turno, la profundidad y los valores de alfa y beta. Primero se revisa si el estado, si la profundidad llegó a cero o si no hay acciones disponibles. Independientemente del caso, se retorna el resultado de la función de evaluación correspondiente.

MAX, recorre las acciones disponibles y se guarda la que tenga el mayor valor y actualiza alfa con el mejor valor encontrado. Si este es mayor o igual que beta, se corta la rama y no se accede a la información dentro de la misma. Para MIN se hace el mismo proceso, pero buscando el menor valor, actualizando beta y cortando cuando el valor es menor o igual que alfa. Esto permite evitar la exploración de ramas que no cambiarían la decisión final, no aportan información útil. El proceso hace la búsqueda más efectiva.

A medida que nodes_evaluated va aumentando, se cambia al siguiente agente y, al explorar un movimiento, la profundidad=profundidad-1. Las acciones se revisan en el orden dado y, si dos tienen el mismo valor, se conserva la primera. Finalmente, se inicia la búsqueda con alfa en menos infinito y beta en más infinito, y se retorna la mejor acción encontrada.

5.2 Diseño de evaluation_function

Para la función de evaluación, primero se revisa si el estado es Win or Lose. Si lo es, se retorna base_evaluation_function.

Para los demás estados, se escogió usar el puntaje actual y restarle un castigo por los terminales pendientes, la distancia al terminal más cercano y la cercanía del intruso. Por cada terminal pendiente se restan 80 puntos y por cada unidad de distancia al objetivo se restan 10. Esto busca que el defensor prefiera completar los terminales y acercarse a los que todavía faltan.

Se calcula el riesgo como $120 / (\text{distancia_intruso} + 1)$. Como la distancia está en el denominador, entre más cerca esté el intruso, mayor será el castigo. Las distancias se calculan con state.layout.distance, para tener en cuenta las paredes.

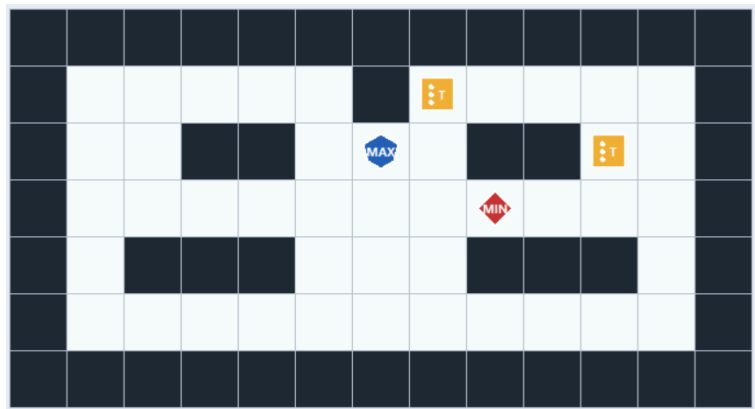
Si no hay terminales pendientes, la distancia al objetivo se deja en cero. Si ninguno de los pendientes es alcanzable, se usa el ancho por la altura del mapa como valor de distancia. Por otro lado, si el intruso no es alcanzable, el riesgo se deja en cero. Finalmente, se limita el resultado entre -999 y 999 para cumplir con el rango solicitado para los estados no terminales.

5.3 Equivalencia y comparación de nodos

Para las mismas condiciones se comparan Minimax y AlfaBeta:

dual_terminal, Profundidad 4.

Inicio:



Minimax:

A 12x12 grid representing a game board. The board has a dark blue border. The interior is a 10x10 grid of light blue and dark blue squares. A blue hexagon with 'MAX' is at (5, 5). A red diamond with 'MIN' is at (7, 7). Two orange squares with 'T' are at (5, 6) and (8, 8). The board is symmetric about the center.

RED CENTINELA

EJECUCIÓN
MinimaxAgent
Defensor MAX: South
Fotograma 12/12 · Acción inicial: East

MÉTRICAS DEL FOTOGRAMA

Profundidad	4 plies
Rondas	5
Terminales pendientes	0
Puntaje	1000.0
Nodos MAX	1024
Nodos MIN	293

DETALLE DEL PASO
RESULTADO FINAL
Resultado: victoria
Finalización: activación completa

LEYENDA

- MAX Defensor MAX
- MIN Intruso MIN
- T Terminal pendiente
- OK Terminal activada por MAX
- Último movimiento

ESTADO ACTUAL DE LA MISIÓN

ÚLTIMO TURNO
MAX
agente

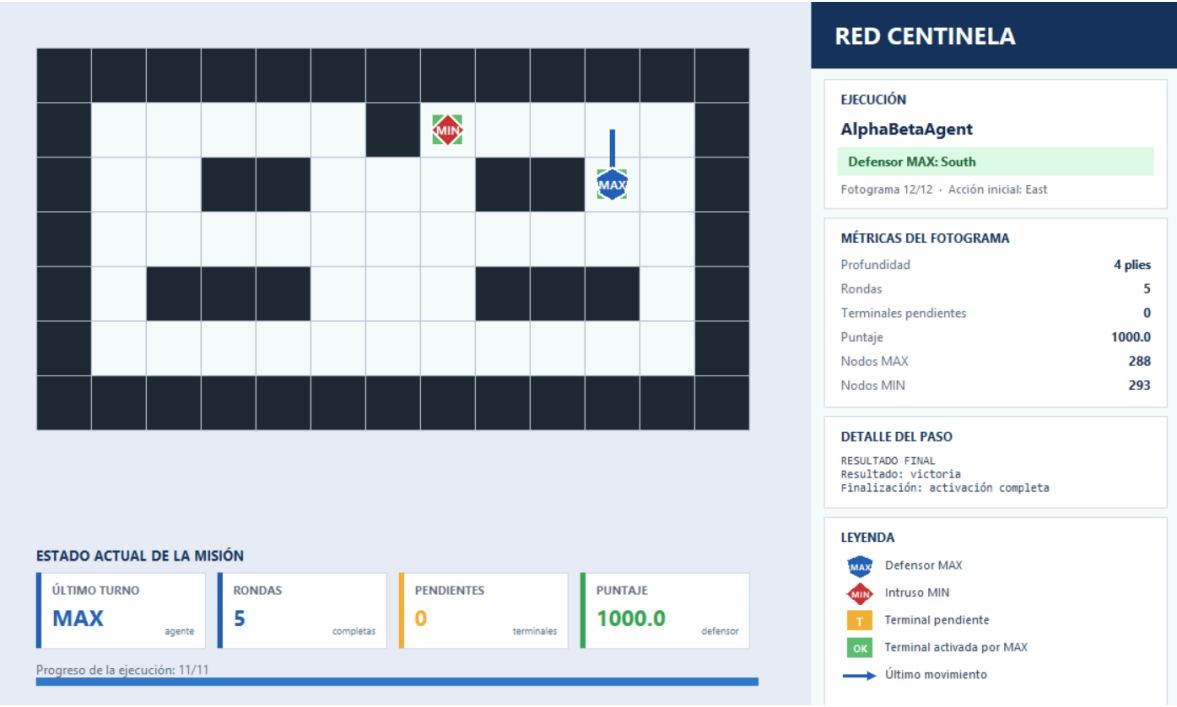
RONDAS
5
completas

PENDIENTES
0
terminales

PUNTAJE
1000.0
defensor

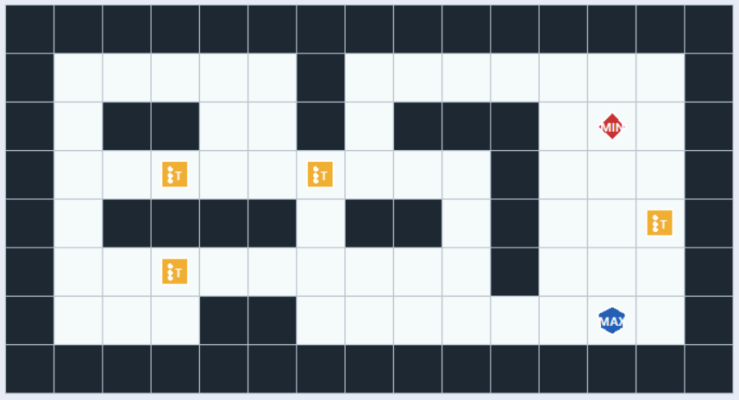
Progreso de la ejecución: 11/11

AlfaBeta:

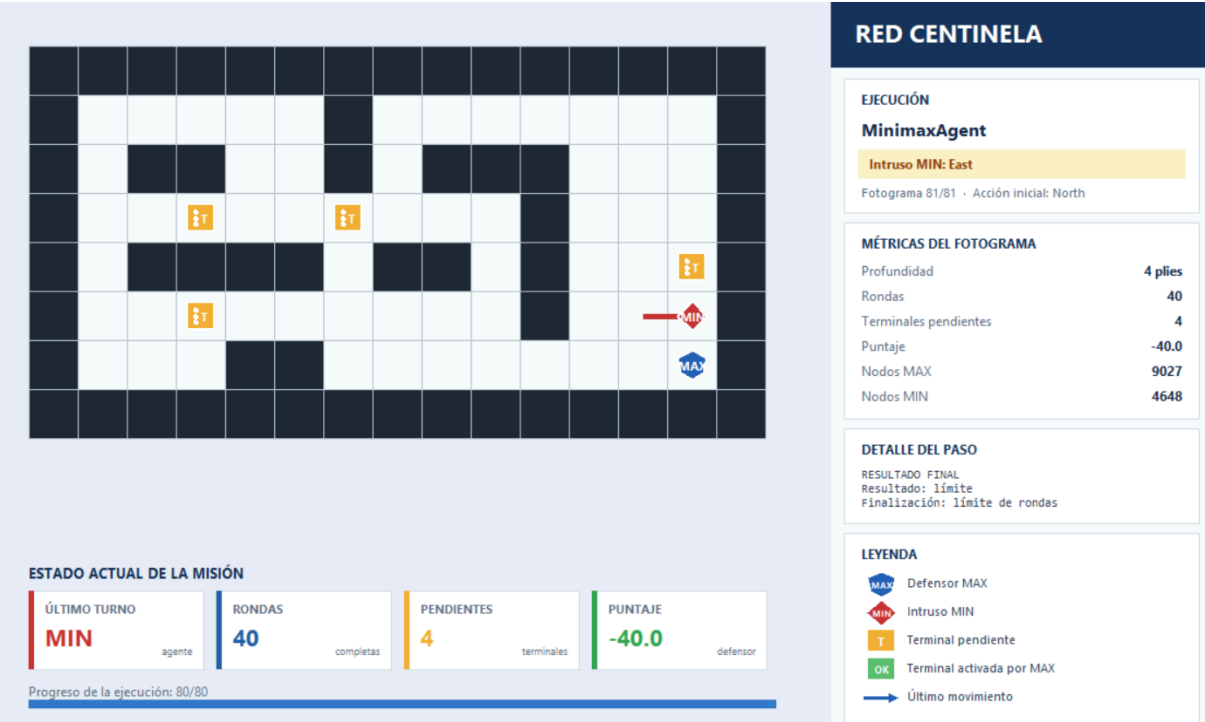


medium_defense, Profundidad 4.

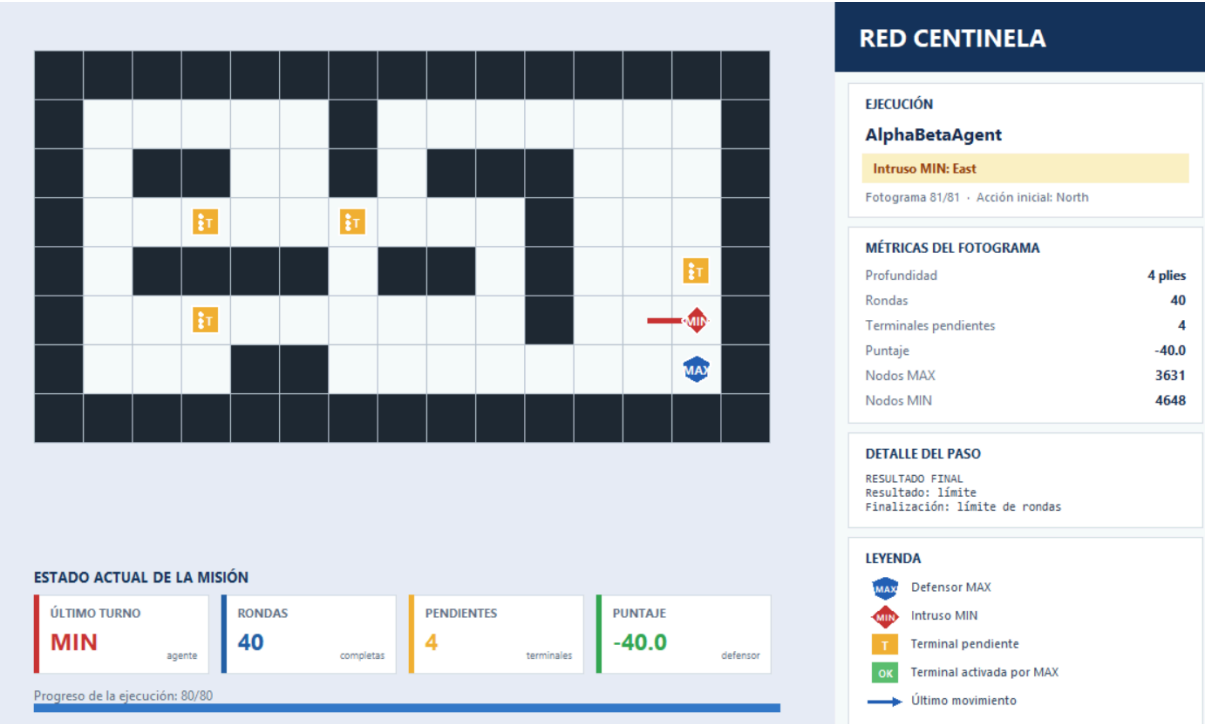
Origen:



Minimax:



AlfaBeta:



En los dos mapas ambos agentes siguen exactamente el mismo camino, y llegan al mismo estado final. Esto se debe al funcionamiento e implementación del AlfaBeta agent. Este agente solo busca reducir la búsqueda de un minimax normal, por lo que la única diferencia entre ellos tiene que ver

con la cantidad de nodos consultados. Nuestros resultados soportan nuestra conclusión. En ambos mapas, la cantidad de nodos consultados por AlfaBeta es menor que a la cantidad de nodos consultada por Minimax.

Nodos consultados por Max en

Dual Terminal: Minimax: 1024, AlfaBeta: 288.

Medium Defense: Minimax: 9027, AlfaBeta: 3831.

AlfaBeta consulta menos por su algoritmo de poda.

¿Por qué esto no ocurre con MIN?

Porque el movimiento de min depende de una función `_intruder_action` aparte. Esta función implementa AlfaBeta, por lo que no importa qué algoritmo utilicemos para el defensor, los nodos de MIN van a ser los mismos.

5.4 Justificación de función de evaluación

Antes de desarrollar `evaluation_function`, se diseñó una heurística que asigna pesos a las diferentes variables que se encuentran en el problema. Se tomaron como base las indicaciones del punto 5.b, en el que nos recomiendan tener en cuenta el puntaje acumulado, las terminales pendientes, la distancia del defensor al objetivo, la distancia del intruso al defensor, el riesgo inmediato de captura y la movilidad disponible.

Escogimos principalmente el puntaje acumulado, el número de terminales pendientes, la distancia a la terminal más cercana y la distancia al intruso. De ahí sacamos la siguiente fórmula.

$$h(x) = \text{puntaje} - 80(\text{terminales_pendientes}) - 10(\text{distancia_terminal_cercana}) - 120/(\text{distancia_intruso} + 1)$$

Puntaje:

Decidimos incluir el puntaje en la heurística porque es un indicador de la cantidad de avances realizados hasta determinado punto. Suma 100 puntos al activar una terminal, y resta un punto por cada acción desarrollada por Max. Lo anterior promueve la activación de terminales y la minimización de movimientos. Además, el cálculo del puntaje ya estaba implementado en el proyecto.

Terminales pendientes:

Es el segundo peso más alto (80), pero el más determinante. Para poder ganar se tienen que activar todas las terminales, por lo que tener varias terminales desactivadas es indicativo de que todavía se está lejos de llegar a la meta y, por lo tanto, la heurística en dicho momento es muy baja. Cuando se activa una terminal, disminuye el número de terminales pendientes, el castigo es menor y aumenta su valor por puntaje (el cual premia esta acción). Se atribuye una gran recompensa al cumplimiento de objetivos.

Distancia terminal más cercana:

Asignamos una penalización de 10 por cada paso de distancia a la terminal pendiente más cercana. Esto permite reconocer el progreso hacia un objetivo incluso antes de activarlo. Sin embargo, escogimos un peso menor que el de las terminales pendientes, porque estar cerca de una terminal no significa estar cerca de la meta general. Si solo se considerara la cercanía, el estado podría parecer más favorable de lo que realmente es. Por eso combinamos ambos factores: la distancia orienta el siguiente movimiento y las terminales pendientes reflejan cuántos objetivos quedan por cumplir. El peso de 80 busca dar más importancia a completar un objetivo que a acercarse unos pocos pasos a él.

Distancia al intruso:

Para este peso utilizamos algo distinto, dividimos el peso entre la variable. Utilizamos esta variable porque el resto no sirve de nada si no tomamos en cuenta si el invasor está cerca de atrapar al defensor en nuestra heurística. Este factor determina qué tan cerca se está de perder. Si la distancia al intruso incrementa, la penalización disminuye (esto por la manera en la que planteamos la fórmula. La división hace que entre más grande sea el denominador, el resultado va a ser menor). Utilizamos el +1 para evitar dividir entre 0. Aunque sea el mayor peso, la única situación en la que este va a ser más determinante que las terminales pendientes es solo si la distancia entre invasor y defensor es 0. Lo anterior nunca puede ocurrir porque apenas se llegue a este estado, se habrá perdido; el invasor está en la misma casilla que el defensor. En caso de que la distancia al intruso sea de una casilla, la fórmula queda $120/(1 + 1) = 60$ y, por lo tanto, menos determinante que una terminal pendiente.

6. Análisis

6.1 Evidencia comparativa de optimización

Algoritmo	Layout	Mejor puntaje	Puntaje medio	Desv. est.	Evaluaciones medias	Iteraciones/ Generaciones	Sensibilidad observada
Ascenso de colina	risk_corridor	116.00	110.60	3.38	153.0	2.8	Alta (107–116)
Ascenso de colina	balanced_city	283.00	280.60	1.62	241.0	3.0	Baja (278–283)
Recocido simulado	risk_corridor	116.00	115.80	0.45	751.0	750.0	Baja (115–116)
Recocido simulado	balanced_city	283.00	282.60	0.89	751.0	750.0	Baja (281–283)
Algoritmo genético	risk_corridor	116.0	114.80	1.47	4040.0	100	Media (113–116)

Algoritmo genético	balanced_city	283.00	281.80	0.98	9060.0	150	Baja (281-283)
--------------------	---------------	--------	--------	------	--------	-----	----------------

Figura 10. Ejecuciones de hill climbing en risk_corridor y balanced_city, usadas para construir la tabla comparativa del punto 6.1.

```

=== TABLA 7.1 - ASCENSO DE COLINA (n=5, seed 7) ===

risk_corridor:
hillClimbing | mejor=116.00 | promedio=110.60 | desviación=3.38 | evaluaciones_promedio=153.0 | tiempo_promedio=0.001440s

balanced_city:
hillClimbing | mejor=283.00 | promedio=280.60 | desviación=1.62 | evaluaciones_promedio=241.0 | tiempo_promedio=0.002816s

=== DETALLE POR SEMILLA - risk_corridor ===
semilla 7 -> Puntaje=108.00 | Evaluaciones=121 | Iteraciones=2
semilla 8 -> Puntaje=116.00 | Evaluaciones=241 | Iteraciones=5
semilla 9 -> Puntaje=107.00 | Evaluaciones=121 | Iteraciones=2
semilla 10 -> Puntaje=113.00 | Evaluaciones=121 | Iteraciones=2
semilla 11 -> Puntaje=109.00 | Evaluaciones=161 | Iteraciones=3

=== DETALLE POR SEMILLA - balanced_city ===
semilla 7 -> Puntaje=278.00 | Evaluaciones=181 | Iteraciones=2
semilla 8 -> Puntaje=281.00 | Evaluaciones=301 | Iteraciones=4
semilla 9 -> Puntaje=281.00 | Evaluaciones=301 | Iteraciones=4
semilla 10 -> Puntaje=283.00 | Evaluaciones=181 | Iteraciones=2
semilla 11 -> Puntaje=280.00 | Evaluaciones=241 | Iteraciones=3
o isabelaarchbold@Isabelas-MacBook-Air-5 RedCentinela %

```

Figura 11. Ejecuciones del algoritmo Recocido simulado en risk_corridor y balanced_city, usadas para construir la tabla comparativa del punto 6.1.

```

>>> python main.py -m optimization -a simulatedAnnealing -l balanced_city --seed 2 --temperature 30 --cooling 0.99 --iterations 750 -q
>>> python main.py -m optimization -a simulatedAnnealing -l balanced_city --seed 3 --temperature 30 --cooling 0.99 --iterations 750 -q
>>> python main.py -m optimization -a simulatedAnnealing -l balanced_city --seed 4 --temperature 30 --cooling 0.99 --iterations 750 -q
>>> python main.py -m optimization -a simulatedAnnealing -l balanced_city --seed 5 --temperature 30 --cooling 0.99 --iterations 750 -q
simulatedAnnealing | mejor=283.00 | promedio=283.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.034321s
simulatedAnnealing | mejor=283.00 | promedio=283.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.041297s
simulatedAnnealing | mejor=283.00 | promedio=283.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.035120s
simulatedAnnealing | mejor=281.00 | promedio=281.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.036185s
simulatedAnnealing | mejor=283.00 | promedio=283.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.042013s

PS C:\Users\davfo\ISIS1611-Taller2-G3\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l risk_corridor --seed 1 --temperature 30 --cooling 0.99 --iterations 750 -q
simulatedAnnealing | mejor=115.00 | promedio=115.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.017651s
PS C:\Users\davfo\ISIS1611-Taller2-G3\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l risk_corridor --seed 2 --temperature 30 --cooling 0.99 --iterations 750 -q
simulatedAnnealing | mejor=116.00 | promedio=116.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.030381s
PS C:\Users\davfo\ISIS1611-Taller2-G3\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l risk_corridor --seed 3 --temperature 30 --cooling 0.99 --iterations 750 -q
simulatedAnnealing | mejor=116.00 | promedio=116.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.024930s
PS C:\Users\davfo\ISIS1611-Taller2-G3\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l risk_corridor --seed 4 --temperature 30 --cooling 0.99 --iterations 750 -q
simulatedAnnealing | mejor=116.00 | promedio=116.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.025474s
PS C:\Users\davfo\ISIS1611-Taller2-G3\RedCentinela> python main.py -m optimization -a simulatedAnnealing -l risk_corridor --seed 5 --temperature 30 --cooling 0.99 --iterations 750 -q
simulatedAnnealing | mejor=116.00 | promedio=116.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.024213s

```

Figura 12. Ejecuciones del algoritmo genético en risk_corridor y balanced_city, usadas para construir la tabla comparativa del punto 6.1.

```

=== TABLA 7.1 - ALGORITMO GENETICO (n=5, seed 7) ===

risk_corridor:
geneticAlgorithm | mejor=116.00 | promedio=114.80 | desviación=1.47 | evaluaciones_promedio=4040.0 | tiempo_promedio=0.077475s

balanced_city:
geneticAlgorithm | mejor=283.00 | promedio=281.80 | desviación=0.98 | evaluaciones_promedio=9060.0 | tiempo_promedio=0.225329s

=== DETALLE POR SEMILLA - risk_corridor ===
semilla 7 ->
Puntaje=116.00 |
Evaluaciones=4040 |
Iteraciones=100
semilla 8 -> Puntaje=113.00 |
Evaluaciones=4040 |
Iteraciones=100
semilla 9 -> Puntaje=113.00 |
Evaluaciones=4040 |
Iteraciones=100
semilla 10 -> Puntaje=116.00 |
Evaluaciones=4040 |
Iteraciones=100
semilla 11 -> Puntaje=116.00 |
Evaluaciones=4040 |
Iteraciones=100

=== DETALLE POR SEMILLA - balanced_city ===
semilla 7 -> Puntaje=281.00 |
Evaluaciones=9060 |
Iteraciones=150
semilla 8 -> Puntaje=281.00 |
Evaluaciones=9060 |
Iteraciones=150
semilla 9 -> Puntaje=283.00 |
Evaluaciones=9060 |
Iteraciones=150
semilla 10 -> Puntaje=283.00 |
Evaluaciones=9060 |
Iteraciones=150
semilla 11 -> Puntaje=281.00 |
Evaluaciones=9060 |
Iteraciones=150

```

6.2 (a) Características de los layouts que explican las diferencias

Los resultados muestran que el comportamiento de los algoritmos depende de las características del layout y no únicamente del algoritmo utilizado. En *local_peak*, el comportamiento está determinado por la presencia de máximos locales. El ascenso de colina depende fuertemente del punto de inicialización, ya que solo acepta movimientos que mejoran la solución actual. Por esta razón, puede quedar atrapado en un máximo local aunque exista una solución mejor. Los algoritmos recocido simulado y genético cuentan con mecanismos de exploración que permiten reducir esta dependencia de la inicialización. En este layout, la diferencia entre algoritmos debe interpretarse principalmente en términos de capacidad de exploración.

En *plateau_grid*, el problema principal está relacionado con las regiones de igual puntaje. El ascenso de colina requiere una mejora estricta para cambiar de estado, por lo que puede permanecer en una solución cuando los vecinos tienen el mismo valor. El recocido simulado puede atravesar estas regiones al aceptar movimientos que no representan una mejora inmediata, mientras que el algoritmo genético explora simultáneamente diferentes soluciones de la población. Por tanto, las diferencias observadas entre los algoritmos reflejan principalmente su capacidad para explorar regiones del espacio de búsqueda con estructuras de meseta.

En *risk_corridor*, los resultados permiten observar con mayor claridad la diferencia en estabilidad entre los algoritmos. Los tres alcanzan el mismo máximo de 116.00, pero el ascenso de colina obtiene una media de 110.60 con una desviación estándar de 3.38, mientras que el recocido simulado obtiene una media de 115.80 con una desviación estándar de 0.45 y el algoritmo genético una media de 114.80

con una desviación estándar de 1.47. Esto indica que el recocido simulado llega al máximo con mucha mayor consistencia.

En `balanced_city`, los resultados son más estables para los tres algoritmos. El máximo observado es 283.00 para todos, mientras que las medias son 280.60 para ascenso de colina, 282.60 para recocido simulado y 281.80 para el algoritmo genético. Las desviaciones estándar son 1.62, 0.89 y 0.98, respectivamente. En este caso, las diferencias entre los algoritmos son menores que en `risk_corridor`, lo que sugiere que la estructura del layout facilita alcanzar soluciones cercanas al máximo.

6.3 (b) Efecto de inicialización, aleatoriedad y parámetros

Los experimentos muestran que la inicialización y la aleatoriedad tienen efectos diferentes dependiendo del algoritmo. En ascenso de colina, la semilla determina la configuración inicial y puede cambiar el resultado final porque el algoritmo solo acepta mejoras estrictas. Esto se observa especialmente en `local_peak`, donde las semillas 7, 12 y 33 terminan en 96.00, 81.00 y 81.00, respectivamente. La semilla 33 no realiza ninguna iteración porque ninguno de sus vecinos mejora el puntaje actual. En `plateau_grid`, varias semillas llegan a 92.00, mientras que la semilla 100 alcanza 94.00, mostrando nuevamente que el punto de inicio puede determinar la región en la que termina la búsqueda.

En recocido simulado, la aleatoriedad también influye porque en cada iteración se selecciona un vecino aleatoriamente y, cuando el movimiento empeora el puntaje, su aceptación depende de una probabilidad asociada a la temperatura. Por esta razón, la configuración de los parámetros T_0 y α afecta el comportamiento del algoritmo. Con $T_0=33$ y $\alpha=0.99$, el puntaje medio fue 115.80 y la desviación estándar 0.45, mientras que con $T_0=5$ y $\alpha=0.90$ el promedio fue 113.20 y la desviación estándar 3.27. La primera configuración, al enfriarse más lentamente, favoreció una exploración más prolongada y produjo resultados más consistentes.

En el algoritmo genético, la población inicial también introduce aleatoriedad, pero el número de generaciones cambia cuánto tiempo tiene la población para evolucionar. Manteniendo fija la población en 60 individuos, la mutación en 0.05 y el elitismo en 2, pasar de 1 a 8 generaciones aumentó el promedio de 276.50 a 281.88 y redujo la desviación estándar de 4.56 a 1.17. Sin embargo, aumentar de 8 a 150 generaciones no produjo una mejora adicional: ambas configuraciones tuvieron un mejor puntaje de 283.00 y un promedio de 281.88. Esto indica que, para este experimento, después de pocas generaciones las iteraciones adicionales tuvieron poco efecto sobre la calidad obtenida.

Por último, los experimentos de Minimax muestran que la profundidad también es un parámetro determinante. En `bottleneck`, con profundidad 2 se seleccionó Stop y la partida terminó en derrota, mientras que con profundidad 4 se seleccionó West y se obtuvo una victoria. Por tanto, aumentar la profundidad permitió considerar consecuencias que estaban fuera del horizonte de la búsqueda más superficial, aunque con un aumento considerable en el número de nodos evaluados.

6.4 (c) Compromiso calidad vs. costo computacional

Usando la tabla 6.1, es posible comparar los algoritmos. Se puede ver claramente que hay una relación entre calidad, consistencia y costo computacional. Por ejemplo, Ascenso de Colina es el más barato para ambos layouts, ya que en risk_corridor hace 153 y en balanced_city hace 241, pero de todos modos es el más sensible a la semilla inicial (por su desviación estándar de 3.38 y 1.62).

Por otro lado, Recocido Simulado tiene un costo intermedio entre los tres algoritmos y hace las mismas evaluaciones para ambos layouts (751), logrando el mismo puntaje máximo de 116.00 y 283.00 y por su desviación estándar de solo 0.45 y 0.89 resulta ser el más consistente de los tres.

Finalmente, el Algoritmo Genético es el más costoso con diferencia, haciendo hasta x12 evaluaciones más que Recocido Simulado y x37 más que Ascenso de Colina, teniendo una peor consistencia que la de Recocido Simulado (desviación de 1.47 y 0.98) y no supera el puntaje máximo ya alcanzado por los otros dos algoritmos. Mostrando que en realidad la relación entre costo computacional y compromiso de calidad no es lineal, y que mayor costo computacional no se traduce en mejor calidad, como explicamos en 3.4-3.5, donde mostramos que llega un punto donde las generaciones adicionales dejan de aportar y solamente se traducen en mayor costo.

Debe tenerse en cuenta que las configuraciones del algoritmo genético no fueron idénticas entre los dos layouts, por lo que las evaluaciones y generaciones no deben utilizarse para hacer una comparación directa de eficiencia entre risk_corridor y balanced_city. Sí permiten, en cambio, comparar el comportamiento de cada configuración dentro del experimento correspondiente.

6.5 (d) Caso donde la profundidad cambia la decisión

Efecto horizonte:

El efecto horizonte se vio claramente en Bottleneck. Con una profundidad de 2, Minimax seleccionó Stop y la partida terminó en derrota por intercepción. El peligro quedaba más allá del horizonte de 2 plies, por lo que la función de evaluación sólo podía valorar el estado del corte sin observar las consecuencias posteriores. Con una profundidad de 4, el algoritmo se anticipó a la mayor parte de la respuesta adversaria y cambió su decisión a West, obteniendo la victoria. Por último, la función de evaluación permitió estimar los estados no terminales a partir del puntaje, las terminales pendientes, la distancia y la proximidad del intruso. Esta estimación no sustituye por completo la información obtenida al explorar más niveles; por esto, ampliar el horizonte cambió la decisión.

6.6 (e) Minimax = alfa-beta, profundidad y orden de acciones

Si AlfaBeta y Minimax utilizan el mismo criterio, estado inicial, profundidad, función de evaluación y orden de acciones, deben producir la misma decisión, conservando la primera acción en los empates. Esto se debe a cómo funciona la poda AlfaBeta. Este algoritmo busca reducir la cantidad de nodos revisados en un Minimax convencional, con el propósito de reducir el costo computacional de la búsqueda. Está diseñado para no omitir ramas que puedan cambiar la decisión final; solo omite la revisión de aquellas que sabe que no pueden modificarla.

Sabiendo esto, podemos afirmar que, aunque AlfaBeta revise menos nodos, solo evita revisar información que no va a cambiar el resultado. Esto implica que se obtiene la misma decisión que con Minimax, con la diferencia de que la cantidad de nodos revisados puede ser menor o igual.

7. Reflexión sobre el uso de IA

7.1. Isabela Archbold

- Proceso de desarrollo y uso de IA

Prompt 1: Acaba de hacer `configuration_score`. Retorna coverage - redundancy - exposure. La primera se suma y las otras dos se restan? Explicame si estoy bien.

Prompt 2: Ahora tengo que hacer `hill_climbing`. Si ya tengo `problem.neighbors(current)`, qué tengo que hacer con todos esos vecinos? Estoy confundida cómo hacer las comparaciones $\geq$.

Prompt 3: Estoy confundida con `iterations` y `evaluations`. Si reviso todos los vecinos y ninguno mejora, ¿eso cuenta como una iteración? y cada vecino que evalúo cuenta como una evaluación?

Prompt 4: Esta es la función que hice. ¿Me puedes decir qué está mal y por qué? No quiero que me la cambies simplemente, quiero entender el error.

Primera versión del código:

```
current = initial_configuration

current_score = configuration_score(problem, current)

history_configurations = [current]

history_scores = [current_score]

evaluations = 1

iterations = 0

while iterations < max_iterations:

    neighbors = problem.neighbors(current)

    best_neighbor = current

    best_score = current_score

    for neighbor in neighbors:

        score = configuration_score(problem, neighbor)

        evaluations += 1

        if score > best_score:
```

```

        best_neighbor = neighbor

        best_score = score

    if best_neighbor == current:

        break

    current = best_neighbor

    current_score = best_score

    history_configurations.append(current)

    history_scores.append(current_score)

    iterations += 1

```

Prompt 5: ¿Cómo puedo probar si diferentes configuraciones iniciales producen resultados diferentes? ¿Qué semillas podría usar para probarlo?

Respuesta: Cambiar seed equivale a cambiar el punto de partida, y como el algoritmo es determinista, una misma semilla produce siempre el mismo resultado. Corre `--seed 7`, `--seed 12` y `--seed 33` en `local_peak` y en `plateau_grid`, guardando las salidas, y busca dos evidencias distintas: semillas que terminen en puntajes diferentes, lo que muestra sensibilidad a la inicialización, y alguna que quede por debajo del mejor valor observado, lo que muestra la detención en un óptimo local. Si en un layout todas las semillas terminan en el mismo valor, eso todavía no prueba que ese valor sea el óptimo: puede ser una meseta de la que el algoritmo no logra salir, así que conviene probar semillas adicionales —por ejemplo 50, 77 y 100— antes de concluir nada.

Prompt 6: Mira los resultados de `risk_corridor` y `balanced_city` en la sección 6: Análisis. Necesito hacer que en la terminal se cree una tabla con la evidencia de la ejecución sin tener que hacer muchos screenshots. Dame una idea.

Respuesta: Pega este bloque completo de una: `clear; echo "=== TABLA 7.1 - ASCENSO DE COLINA (n=5, seed 7) ==="; echo; echo "risk_corridor:"; python3 main.py -m optimization -a hillClimbing -l risk_corridor -n 5 --seed 7 -q; echo; echo "balanced_city:"; python3 main.py -m optimization -a hillClimbing -l balanced_city -n 5 --seed 7 -q; echo; echo "=== DETALLE POR SEMILLA - risk_corridor ==="; for s in 7 8 9 10 11; do echo -n "semilla $s » "; python3 main.py -m optimization -a hillClimbing -l risk_corridor --seed $s -t | grep "Puntaje="; done; echo; echo "=== DETALLE POR SEMILLA - balanced_city ==="; for s in 7 8 9 10 11; do echo -n "semilla $s » "; python3 main.py -m optimization -a hillClimbing -l balanced_city --seed $s -t | grep "Puntaje="; done`

Prompt 7: Mira el enunciado. Está bien en el punto 6.1. haber usado estos dos layouts: `risk_corridor` y `balanced_city` para comprar los tres algoritmos?

Respuesta: Sí, y de hecho es la elección correcta. El enunciado manda a Persona 2 a `risk_corridor` y a Persona 3 a `"risk_corridor, balanced_city u otro layout apropiado"`. Son justo los dos que escogiste, así que

P2 y P3 pueden llenar sus filas con corridas que de todos modos tenían que hacer. Y la rúbrica de 6.1 pide "comparar los tres algoritmos con resultados de calidad y costo". Para eso lo que importa no es cuál layout, sino que las tres filas sean del mismo layout. Con dos layouts y tres algoritmos tienes seis filas comparables por pares, que es más de lo que pide el mínimo.

- Decisiones tomadas a partir de la IA

El uso de IA me ayudó principalmente a encontrar errores de lógica y a revisar si mi implementación correspondía con lo que pedía el problema. Algunas correcciones fueron de detalle, como los nombres de los campos de OptimizationResult, pero otras fueron de fondo. Por ejemplo, tuve que cambiar la forma de calcular el puntaje porque estaba sumando componentes que en realidad funcionan como penalizaciones. También tuve que entender que una iteración debía representar una mejora aceptada y no solamente una revisión de vecinos.

Al principio me costaba formular prompts útiles porque todavía no tenía claro qué parte del código estaba fallando. Mis primeras preguntas fueron más generales y, a medida que entendía mejor el problema y revisaba el código, pude hacer preguntas más específicas. Esto hizo que la IA fuera más útil como herramienta para revisar mi razonamiento y no solamente para generar código. También me ayudó a optimizar tareas repetitivas, como poner una a una en la terminal los resultados de la tabla comparativa entre layouts.

7.2. Juan David Forero

Prompt 1: estoy realizando una función en python para que haga el algoritmo de recocido simulado. Esta función se apoya de una función auxiliar de enfriamiento. Te adjuntare el código que realicé basado en la teoría y el pseudo código que adjunto el cual corresponde al visto en mi clase.

Respuesta: Sí. Con el código que muestras y el material de clase, se pueden identificar claramente 4 errores principales en tu implementación. Según las diapositivas, el recocido simulado debe hacer cuatro cosas en cada iteración: escoger un vecino aleatorio, calcular el cambio de calidad ΔE , aceptar siempre una mejora y aceptar un empeoramiento con una probabilidad que depende de ΔE y de la temperatura. Además, la temperatura debe ir disminuyendo con el tiempo.

Los principales errores son: 1) reinicias la temperatura en cada iteración con `temperature = initial_temperature`, por lo que nunca se aplica el enfriamiento; 2) solo aceptas candidatos que mejoran ($\Delta > 0$), cuando el recocido simulado también debe permitir aceptar soluciones peores con una probabilidad dependiente de Δ y de la temperatura; 3) guardas en el historial el candidate aunque haya sido rechazado, cuando deberías guardar el estado current que realmente quedó después de la decisión; y 4) la actualización de `best_configuration` está planteada sobre el candidato rechazado, aunque este punto depende de cómo esté definido exactamente el pseudocódigo de tu clase.

Código recomendado:

```
while iteration < max_iterations:
```

```
if temperature <= minimum_temperature:

    break

neighbors = problem.neighbors(current)

if not neighbors:

    break

candidate = rng.choice(neighbors)

candidate_score = configuration_score(problem, candidate)

evaluations += 1

delta = candidate_score - current_score

# Si mejora, siempre aceptar

if delta > 0:

    current = candidate

    current_score = candidate_score

# Si empeora, aceptar con cierta probabilidad

else:

    probability = math.exp(delta / temperature)

    r = rng.random()

    if r < probability:

        current = candidate
```

```

        current_score = candidate_score

# Actualizar el mejor estado encontrado

if current_score > best_score:

    best_configuration = current

    best_score = current_score

history.append(current)

score_history.append(current_score)


iteration += 1


temperature = cooling_schedule(

    initial_temperature,

    cooling_rate,

    iteration

)

```

Diferencias con el código definitivo: Al realizar pruebas los resultados tenían ciertas anomalías, por lo cual se modificó el momento en el que se evalúa la nueva temperatura. También se redujeron ciertas líneas de código en código más compacto pero funcional.

Codigo despues de los cambios:

```

def cooling_schedule(initial_temperature: float, cooling_rate: float, iteration: int) » float:

    """

    Retorna el programa geométrico  $T(t) = T_0 * \alpha^{**t}$ .

    """

    return initial_temperature * (cooling_rate ** iteration)

```

```

def simulated_annealing(
    problem: SmartGridOptimizationProblem,
    initial_configuration: Configuration,
    initial_temperature: float = 20.0,
    cooling_rate: float = 0.97,
    max_iterations: int = 500,
    rng: random.Random | None = None,
) → OptimizationResult:
    rng = rng or random.Random()
    minimum_temperature = 1e-9
    current = initial_configuration
    current_score = configuration_score(problem, current)
    evaluations = 1

    best_configuration = current
    best_score = current_score

    history = [current]
    score_history = [current_score]

    temperature = initial_temperature
    iteration = 0

    while iteration < max_iterations:

```

```

temperature = cooling_schedule(initial_temperature, cooling_rate, iteration)

if temperature <= minimum_temperature:

    break

candidate = rng.choice(problem.neighbors(current))

candidate_score = configuration_score(problem, candidate)

evaluations += 1

delta = candidate_score - current_score

if delta > 0 or rng.random() < math.exp(delta / temperature):

    current = candidate

    current_score = candidate_score

    if current_score > best_score:

        best_configuration = current

        best_score = current_score

history.append(current)

score_history.append(current_score)

iteration += 1

return OptimizationResult(

    best_configuration=best_configuration,

    best_score=best_score,

    evaluations=evaluations,

    iterations=iteration,

```

```

history=history,

score_history=score_history,

metadata={

    "initial_temperature": initial_temperature,

    "cooling_rate": cooling_rate,

    "final_temperature": temperature,

},

)

```

7.3. Samuel Zuluaga:

Prompt 1: Escribí mi propia versión de one_point_crossover usando el pseudocódigo del libro que te pase como contexto del proyecto, revisa mi implementación y ayúdame a correr las pruebas en los layouts para ver si funciona adecuadamente.

****La versión del código no cambio, porque estaba bien implementada.**

Prompt 2: Estoy haciendo la funcion de swap_mutation, (aparece en el libro que te mande de contexto), sin embargo no se como especificar los diferentes casos en este problema, y en especifico para los grupos vacíos no se que retornar.

Codigo antes del prompt:

```

if rng.random() < mutation_probability: # Caso donde toca mutar

    indices_unos = []

    indices_ceros = []

    for i, bit in enumerate(individual):

        if bit == 1:

            indices_unos.append(i)

        else:

            indices_ceros.append(i)

    cromosoma = list(individual)

    i = rng.choice(indices_unos)

```

```

        j = rng.choice(indices_ceros)

        cromosoma[i] = 0

        cromosoma[j] = 1

        return tuple(cromosoma)

    else:

        return individual # Caso 2: no toca mutar

```

Respuesta: “El docstring de la función exige explícitamente: "Si alguno de los dos grupos está vacío, no hay un intercambio posible". Esto ocurre cuando el cromosoma es todo ceros (no hay ningún índice activo) o todo unos (no hay ningún índice inactivo) — en ese caso, `rng.choice([])` sobre una lista vacía lanza un `IndexError`. Hay que verificar ambas listas antes de llamar a `rng.choice`, y si alguna está vacía, retornar el individuo sin cambios (igual que el caso de "no toca mutar"). Se agregó la validación inmediatamente después de separar los índices activos e inactivos.” (OpenAI, 2026)

Código corregido:

```

if rng.random() < mutation_probability:

    indices_unos = []

    indices_ceros = []

    for i, bit in enumerate(individual):

        if bit == 1:

            indices_unos.append(i)

        else:

            indices_ceros.append(i)

    if not indices_unos or not indices_ceros:

        return individual # No hay intercambio posible

    cromosoma = list(individual)

    i = rng.choice(indices_unos)

    j = rng.choice(indices_ceros)

```

```

    cromosoma[i] = 0

    cromosoma[j] = 1

    return tuple(cromosoma)

else:

    return individual

```

Prompt 3: Estoy haciendo genetic_algorithm, sin embargo no entiendo cómo llamar a configuration_score o como usar elite_size). El código que te voy a pasar no está completo y la mayoría es pseudocódigo del libro.

Codigo antes de cambios:

```

poblacion = crear_poblacion_inicial(...) # ya dado

mejor_global = mejor(poblacion, key=configuration_score)

for gen in range(generations):

    nueva_poblacion = elite(poblacion, elite_size) # conservar los mejores tal cual
    ...

    mejor_actual = mejor(poblacion, key=configuration_score)

    if configuration_score(mejor_actual) > configuration_score(mejor_global):

        mejor_global = mejor_actual

```

Respuesta; “Había dos confusiones distintas mezcladas en el mismo pseudocódigo:

configuration_score recibe dos argumentos (problem y la configuración), no uno solo — llamarlo como configuration_score(mejor_actual) faltaba el argumento problem.

Para "encontrar el mejor individuo de una población según su puntaje" no se usa configuration_score ni una función mejor() que no existe — se usa la función builtin de Python max(lista, key=...), pasándole una función que calcule el puntaje de cada elemento: max(poblacion, key=lambda config: configuration_score(problem, config)).

elite_size es solo un número (cuántos individuos conservar), no el nombre de una función elite(). Para implementar el elitismo hay que ordenar la población por puntaje (de mayor a menor) y quedarse con los primeros elite_size: sorted(..., key=..., reverse=True)[:elite_size].” (OpenAI, 2026)

Código después de cambios.

```
poblacion = problem.initial_population(population_size, rng)

scores = [configuration_score(problem, config) for config in poblacion]

mejor_global = max(poblacion, key=lambda config: configuration_score(problem,
config))

for gen in range(generations):

    ranked = sorted(zip(poblacion, scores), key=lambda item: item[1], reverse=True)

    nueva_poblacion = [config for config, _ in ranked[:elite_size]]

    ...

    mejor_actual = max(poblacion, key=lambda config: configuration_score(problem,
config))

    if configuration_score(problem, mejor_actual) > configuration_score(problem,
mejor_global):

        mejor_global = mejor_actual
```

Prompt 4: **Intento de arriba. Porfa revisame el código que hice intentando arreglar los errores que me dijiste antes.

Respuesta: “Ya se corrigió el problema de configuration_score/elite del prompt anterior, pero el ... (la parte de reproducción) sigue sin implementarse — ahí falta: seleccionar dos padres con problem.tournament_select(poblacion, scores, rng), cruzarlos con one_point_crossover(padre1, padre2, rng) (con rng como tercer argumento), reparar cada hijo con problem.repair_configuration(hijo, rng), aplicar swap_mutation, y repetir hasta llenar nueva_poblacion. Además, después del ciclo interno falta reemplazar poblacion = nueva_poblacion, recalcular scores sobre la nueva población, contar las evaluaciones acumuladas, y al final retornar un OptimizationResult con los campos reales (best_configuration, best_score, evaluations, iterations, history, score_history) en vez de un ... sin terminar.” (OpenAI, 2026)

Codigo final:

```

poblacion = problem.initial_population(population_size, rng)

scores = [configuration_score(problem, config) for config in poblacion]

evaluations = len(poblacion)


mejor_indice = scores.index(max(scores))

mejor_global = poblacion[mejor_indice]

mejor_global_score = scores[mejor_indice]


history_configurations = [mejor_global]

history_scores = [mejor_global_score]


for gen in range(generations):

    ranked = sorted(zip(poblacion, scores), key=lambda item: item[1], reverse=True)

    nueva_poblacion = []

    for config, score in ranked[:elite_size]:

        nueva_poblacion.append(config)


    while len(nueva_poblacion) < population_size:

        padre1 = problem.tournament_select(poblacion, scores, rng)

        padre2 = problem.tournament_select(poblacion, scores, rng)

        hijo1, hijo2 = one_point_crossover(padre1, padre2, rng)

        hijo1 = problem.repair_configuration(hijo1, rng)

        hijo2 = problem.repair_configuration(hijo2, rng)

        hijo1 = swap_mutation(hijo1, mutation_probability, rng)

        hijo2 = swap_mutation(hijo2, mutation_probability, rng)

```

```

nueva_poblacion.append(hijo1)

if len(nueva_poblacion) < population_size:
    nueva_poblacion.append(hijo2)

poblacion = nueva_poblacion

scores = [configuration_score(problem, config) for config in poblacion]
evaluations += len(poblacion)

mejor_indice = scores.index(max(scores))
mejor_actual = poblacion[mejor_indice]
mejor_actual_score = scores[mejor_indice]

if mejor_actual_score > mejor_global_score:
    mejor_global = mejor_actual
    mejor_global_score = mejor_actual_score

history_configurations.append(mejor_global)
history_scores.append(mejor_global_score)

return OptimizationResult(
    best_configuration=mejor_global,
    best_score=mejor_global_score,
    evaluations=evaluations,
    iterations=generations,

```

```

    history=history_configurations,

    score_history=history_scores,

)

```

7.4. Juan José Castro Maldonado

Prompt 1: Veo que en la funcion_intruder_action se implementa una versión de AlfaBeta. ¿Me serviría hacer una implementación parecida? Ayúdame a plantear una primera versión del código.

Prompt 2: Dado el siguiente código, dime si estoy haciendo líneas de más o si existe una manera de volverlo más compacto (se adjunta la primera versión del código).

Prompt 3: Me preocupa que, en algunos casos, MIN y MAX se queden repitiendo sus movimientos. ¿es normal?

Prompt 4: Para hacer la evaluation_function, ¿debería incluir el puntaje? Si es así, ¿por qué lo recomiendas?

Prompt 5: Aún después de hacer algunas modificaciones en la evaluation_function, en algunos casos, especialmente los de menores profundidades, siguen repitiendo algunas acciones hasta que se acaban los turnos, ¿será que sigue habiendo algún error en esta función ?

Primeras versiones del código:

class AlphaBetaAgent(MultiAgentSearchAgent):

```

def get_action(self, state: GameState):

    self.nodes_evaluated = 1

    if state.is_win() or state.is_lose():

        return None

    acciones = state.get_legal_actions(o)

    if not acciones:

        return None

    mejor_accion = acciones[0]

    mejor_valor = float("-inf")

    alfa = float("-inf")

    beta = float("inf")

    for accion in acciones:

```

```

nuevo_estado = state.generate_successor(o, accion)

valor = self.buscar(nuevo_estado, 1, self.depth - 1, alfa, beta)

if valor > mejor_valor:

    mejor_valor = valor

    mejor_accion = accion

if mejor_valor > alfa:

    alfa = mejor_valor

return mejor_accion

```

def buscar(self, estado, agente, profundidad, alfa, beta):

```

self.nodes_evaluated += 1

if estado.is_win() or estado.is_lose():

    return evaluation_function(estado)

if profundidad == 0:

    return evaluation_function(estado)

acciones = estado.get_legal_actions(agente)

if not acciones:

    return evaluation_function(estado)

if agente == 0:

    mejor_valor = float("-inf")

    for accion in acciones:

        nuevo_estado = estado.generate_successor(o, accion)

        valor = self.buscar(nuevo_estado, 1, profundidad - 1, alfa, beta)

        if valor > mejor_valor:

            mejor_valor = valor

        if mejor_valor > alfa:

            alfa = mejor_valor

    if mejor_valor >= beta:

        break

```

```

else:

    mejor_valor = float("inf")

    for accion in acciones:

        nuevo_estado = estado.generate_successor(1, accion)

        valor = self.buscar(nuevo_estado, 0, profundidad - 1, alfa, beta)

        if valor < mejor_valor:

            mejor_valor = valor

            if mejor_valor < beta:

                beta = mejor_valor

            if mejor_valor <= alfa:

                break

    return mejor_valor

```

def evaluation_function(state: GameState) → float:

```

if state.is_win() or state.is_lose():

    return base_evaluation_function(state)

defensor = state.defender_position

intruso = state.intruder_position

pendientes = state.pending_terminals

distancia_objetivo = float("inf")


for terminal in pendientes:

    distancia = state.layout.distance(defensor, terminal)

    if distancia < distancia_objetivo:

        distancia_objetivo = distancia

if len(pendientes) == 0:

    distancia_objetivo = 0

else:

    if distancia_objetivo == float("inf"):

        distancia_objetivo = (state.layout.width * state.layout.height)

```

```

distancia_intruso = state.layout.distance(defensor, intruso)

if distancia_intruso == float("inf"):

    riesgo = 0

else:

    riesgo = 120 / (distancia_intruso + 1)

cantidad_pendientes = len(pendientes)

penalizacion_pendientes = 80 * cantidad_pendientes

penalizacion_distancia = 10 * distancia_objetivo

valor = valor - penalizacion_pendientes

valor = valor - penalizacion_distancia

valor = valor - riesgo

if valor > 999:

    valor = 999.0

if valor < -999:

    valor = -999.0

return float(valor)

```

Decisiones tomadas a partir de la IA:

Se utilizó la IA para pedir consejos con respecto al largo del código y también para guiar acerca de qué esperar con respecto al comportamiento de las funciones. Para AlfaBeta, la implementación de AlfaBeta agent, el mayor aporte de la IA fue la sugerencia de definir una función dentro de otra para así poder implementar todo el requerimiento en una sola función.

Antes de implementar `evaluation_function`, probé el programa con la `base_evaluation_function`. Noté que usando ese criterio, el algoritmo tendía a entrar en ciclos en los que MAX iba al norte, MIN iba al norte para que luego MAX fuera al sur y MIN fuera al sur. Después de ver esto, le pregunté al chat si este comportamiento era normal, y su respuesta me ayudó a entender la importancia de la `evaluation function` que yo debía implementar. Tenía que hacer una implementación que recompensara a AlfaBeta por estar más cerca de cumplir la meta (ganar) y la castigara por no hacer avances o estar cerca a perder.

Creé mi primera versión de `evaluation_function` y le pedí a chat que me diera sugerencias de cómo la podía mejorar. La IA me recomendó utilizar puntaje dentro de la función de heurística, ya que era un indicador a tomar en cuenta porque minimiza movimientos, premia la activación de terminales y ya estaba implementado. Más allá de eso, estábamos de acuerdo con el resto de variables utilizadas y los pesos asignados; el resto de sugerencias eran con respecto a la longitud del código.

Después de implementar esa versión “final” de `evaluation_function`, decidí probarla con diferentes profundidades para ver su efectividad. Descubrí que si la profundidad era muy baja y el mapa no permitía escoger mucha diversidad de caminos desde el origen, era probable llegar a los bucles que había encontrado cuando probé la `base_evaluation_function`. También descubrí que existen distintos estados específicos que generan ciclos. Un ejemplo de esto es cuando el invasor llega a una terminal desactivada sin querer y es la más cercana a MAX. Por la heurística definida, el defensor buscará llegar a la terminal más cercana, pero a la vez alejarse del invasor, por lo que preferirá conceder su turno sin moverse. Este tipo de ciclos me llegaron a preocupar y, por lo tanto, terminé consultando a la IA para ver si era normal encontrar este tipo de situaciones. Su respuesta fue que la heurística no estaba mal, era posible encontrar este tipo de situaciones, pero que una heurística más completa las puede evitar. Ya con esto en mente, no modifique más la función porque considero que es una heurística suficiente para el problema.

7.5. Isabella Largacha

Prompt 1 : Podrías explicar un poco a es la estructura de Minimax para guiarme de como hacer el código y tener un concepto claro de cómo hacerlo

Prompt 2 : Me podrias ayudar a identificar errores de este código para tener claridad y poder corregirlos para su mejor funcionamiento y guiarme un poco para las pruebas

Prompt 3: Haz una revisión de qué entregables me faltan

Sugerencias principales recibidas

La IA explicó la estructura de Minimax y propuso una función recursiva para alternar MAX y MI corregida de los errores de la versión inicial, indicó cómo manejar los casos terminales y el corte por profundidad y explicó que se debía utilizar una comparación estricta para conservar la primera acción en los empates. También orientó la ejecución de pruebas en `single_terminal` y `bottleneck`.

Versión inicial:

```
class MinimaxAgent(MultiAgentSearchAgent):

    def get_action(self, state: GameState) -> str | None:

        self.nodes_evaluated = 0

        def minimax_value(current_state, agent_index, current_depth):

            self.nodes_evaluated += 1

            if current_state.is_win() or current_state.is_lose():

                return evaluation_function(current_state)

            if current_depth == self.depth:

                return evaluation_function(current_state)
```

```

legal_actions = current_state.get_legal_actions(agent_index)

if len(legal_actions) == 0:

    return 0

next_agent = agent_index + 1

if agent_index == 0:

    best_value = 0

    for action in legal_actions:

        successor = current_state.generate_successor(

            agent_index, action

        )

        value = minimax_value(

            successor,

            next_agent,

            current_depth + 1

        )

        if value > best_value:

            best_value = value

    return best_value

else:

    best_value = 0

    for action in legal_actions:

        successor = current_state.generate_successor(

            agent_index, action

        )

        value = minimax_value(

            successor,

```

```

        0,

        current_depth + 1

    )

    if value < best_value:

        best_value = value

    return best_value

legal_actions = state.get_legal_actions(0)

if len(legal_actions) == 0:

    return None

best_action = None

best_value = 0

for action in legal_actions:

    successor = state.generate_successor(0, action)

    value = minimax_value(

        successor,

        1,

        0

    )

    if value >= best_value:

        best_value = value

        best_action = action

return best_action

```

Verificación

python main.py -m adversarial -a MinimaxAgent -l single_terminal -d 2 -q

python main.py -m adversarial -a MinimaxAgent -l bottleneck -d 2 -q

```
python main.py -m adversarial -a MinimaxAgent -l bottleneck -d 4 -q
```

Decisiones tomadas a partir de la IA

Para desarrollar mi parte utilicé IA como guía para comprender e implementar el algoritmo Minimax. Al revisar sus sugerencias aprendí cómo alternan los jugadores MAX y MIN corriendo los errores que tenía en mi versión inicial también la use como guía de cómo funciona el corte por profundidad y por qué la raíz, los estados terminales y los estados de corte deben incluirse en `nodes_evaluated`. La ayuda me permitió organizar la función recursiva y completar correctamente la implementación también incluyó mejoras secundarias, como conservar la primera acción cuando existen empates y comprobar que únicamente se modificara `MinimaxAgent.get_action`. Al principio fue difícil formular prompts útiles porque todavía no comprendía del todo cómo dividir el algoritmo en casos base como nodos MAX y nodos MIN. Hacer preguntas paso a paso y ejecutar cada prueba hizo que el proceso fuera más claro. Después de comparar las profundidades 2 y 4 en bottleneck, entendí que aumentar la profundidad permite anticipar consecuencias que quedan fuera del horizon aunque incrementa el costo.